
EECS C106B: Project 4a - Grasping *

Due date: March 30th at 11:59pm

Multifingered Grasping using Dexterous Hands in MuJoCo

In this project you will explore grasping techniques on an Allegro multifingered hand using MuJoCo, an open sourced physics engine commonly used to simulate robotics applications. You will be expected to develop the grasping pipeline to get a robotic arm to grasp and pick up a ball in simulation. We will explore techniques such as developing a multifingered inverse kinematics (IK) solver, synthesizing force closure grasps, and grasp evaluation. The learning goals of this lab are to gain experience using research-grade simulation software, gain familiarity with classical grasping and manipulation techniques, and develop the ability to implement theory for a practical application.

Contents

1	Theory	2
1.1	Grasping Intro	2
1.1.1	Problem Setup	2
1.2	Friction	3
1.3	The Grasp Matrix	4
1.3.1	Friction Cone Discretization	5
1.4	Force Closure	5
1.5	Grasp Synthesis Algorithm	6
1.5.1	The Necessary Condition	6
1.5.2	The Sufficient Condition	7
1.5.3	The Algorithm	7
1.5.4	Grasp Synthesis Results	7
1.6	Levenberg-Marquardt Inverse Kinematics	9
1.7	Other Grasping Methods	9
1.7.1	Model-less Algorithms	9
1.7.2	Dual Arm Grasping	9
1.7.3	Machine Learning Approaches	9
2	Project Tasks	10
3	Deliverables	11
4	Getting Started	12
4.1	GitHub	12
4.2	Starter Code	12
4.3	Implementation Tips	13
4.4	Common Issues/Concerns	13
5	Scoring	13
6	Submission	13
7	Improvements	13
8	References	14

*Developed by Yarden Goral and Karim El-Refai, Spring 2025. Adapted by Yarden Goral for Spring 2026

1 Theory

Preface: In robotics there's often a disconnect between theory and implementation. Researchers, especially in the grasping literature, often prove theoretical results that are comprehensive to many geometries, following their mathematical assumptions. Many times, it is not practical or possible to implement these algorithms as is, depending on the task. Thus one of the goals of this project is to practice implementing theory that we've learned in class to achieve a practical result. You might find yourself frustrated that you have to discover certain simplifications and assumptions, resulting in an implementation that is different from the exact math. It might be hard to tell if your theory is actually working. That's part of the learning process. The goal is to learn how to design experiments that reveal what your mathematical implementations are actually doing, which can involve potentially iterating or completely changing your methods. In the following theory sections we discuss some theoretical results in grasping and manipulation, drawing connections from theory to implementation, but there is a lot of room for creativity, and we encourage you to try out your ideas and discuss them with course staff and with each other.

1.1 Grasping Intro

The problem of grasping, or restraining an object using a robotic hand or gripper, has been one of the most difficult and well-researched topics in robotics. The difficulty of grasping is captured with Moravec's paradox, "It is comparatively easy to make computers exhibit adult level performance on intelligence tests or playing checkers, and difficult to give them the skills of a one-year old when it comes to perception and dexterity." In the following sections we will walk through some common theory useful for grasping, and for this project in particular. Much of the theory in this doc is drawn from [1] and [2].

1.1.1 Problem Setup

Our setup consists of an Allegro multifingered hand attached to the end effector of a Sawyer arm. We can think of our joint configuration, $\mathbf{q}$ as a concatenation of two components:

$$\mathbf{q} = \begin{bmatrix} \mathbf{q}_a \\ \mathbf{q}_h \end{bmatrix},$$

Where $\mathbf{q}_a \in \mathbb{R}^7$ represents the joint configuration for the arm, and $\mathbf{q}_h \in \mathbb{R}^{16}$ is the joint configuration for the hand. Figure 2 shows a graphic of this setup. While figure 1 shows a graphic of the allegro hand setup

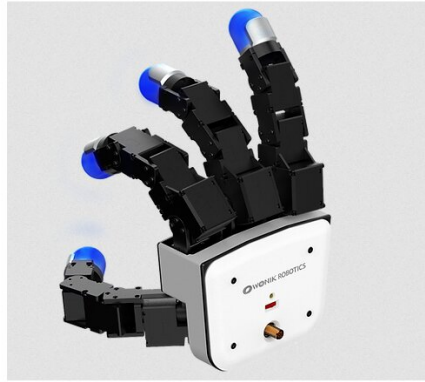


Figure 1: Allegro Hand

When working with contact forces, we must be comfortable with wrenches. A wrench in 3D at contact i is the six-dimensional vector:

$$\mathbf{w} = \begin{bmatrix} \mathbf{f} \\ \boldsymbol{\tau} \end{bmatrix},$$

Where $\mathbf{f} \in \mathbb{R}^3$ is the force vector on a contact point, and $\boldsymbol{\tau} \in \mathbb{R}^3$ is the torque vector.

Figure 3 shows an example contact between a finger and an object. We define F_{c_i} to be the applied wrench of contact i represented in the contact frame. Much of grasping theory deals with where these force vectors are and how friction plays a role in the grasp. The following sections discuss this in detail.

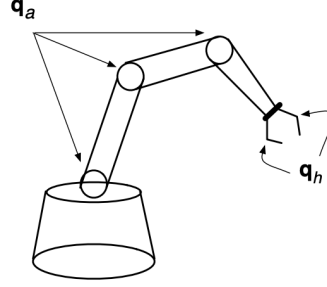


Figure 2: The joint configuration of our setup

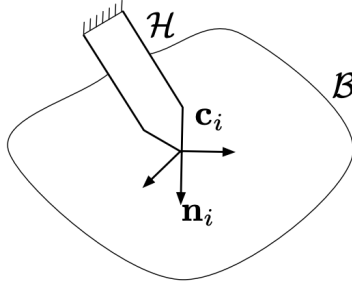


Figure 3: The i th finger making contact, denoted as c_i , at for hand $\mathbf{H}$ with a body $\mathbf{B}$ (surface of an object). Normal force, n_i , points inward and is perpendicular to the tangent plane of the surface at that point.

1.2 Friction

Contact points are usually characterized by one of three friction approximations:

Frictionless contact point: The finger only exerts a force along $\mathbf{n}_i$, the normal to the surface at the contact point. Conceptually, this means that the finger can push but not pull. The constraint for this contact is $f_n \geq 0$, where f_n is the force along the normal.

Frictional contact point: The finger can exert a force along the normal to the object surface. In this class we use Coulomb's law, which states that the point of contact on the finger has no motion as long as any tangential component to the force vector is less than or equal to the product of the normal force and the coefficient of static friction between the finger and the object:

$$f_t \leq \mu f_n$$

For frictionless point contacts, we can represent the applied wrench as:

$$F_{c_i} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} f_{c_i}, f_{c_i} \geq 0$$

Where $f_{c_i} \in \mathbb{R}$ is the magnitude of friction applied by the finger in the normal direction.

Where f_t and f_n are the tangential and normal components of the force vector respectively, and μ is the coefficient of static friction between the finger and the object.

Since f_t can have any orientation, it's common to model the set of possible contact forces using a friction cone. Friction cones originate from a contact point and are tangential to the object surface. Its aperture is $\tan^{-1}\mu$ and f_t is usually expressed in terms of two projections f_1 and f_2 along the axis defined on the tangent plane. Figure 4 details a friction cone example. It is defined as $f_t = \sqrt{f_1^2 + f_2^2}$. We still have the constraint that $f_n \geq 0$. Using this, we define a friction cone at contact point $\mathbf{p}_i$ as:

$$FC(\mathbf{p}_i) = \{[f_1 f_2 f_3]^T \in \mathbb{R}^3 \mid \sqrt{f_1^2 + f_2^2} \leq \mu f_n \wedge f_n \geq 0\}$$

As long as the total force exerted by each finger is within this cone, the finger should not slip. In this case, We represent the wrench applied to the object with respect to a basis of directions which are consistent with the friction model:

$$F_{c_i} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} f_{c_i}, f_{c_i} \in FC_{c_i}$$

We define the above matrix as $B_i \in \mathbb{R}^{p \times m_i}$, representing the wrench basis for contact i . Here, p is the dimension of the wrench, which in 3D will be 6, and m_i , indicates the number of independent forces that can be applied by the contact.

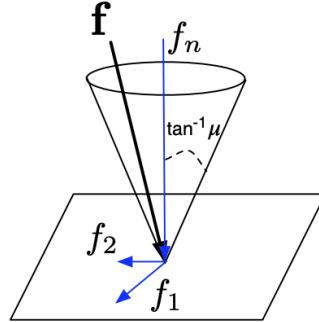


Figure 4: Friction Cone

Soft contact point: This is the case where the finger can exert twisting, or torsional force about the normal of the contact point. Here we can define torsional intensity, $\tau \leq \gamma f_n$, where $\gamma > 0$ is the coefficient of torsional friction. In this lab, we will initially assume point contacts (no torsion) for our fingers, but it's possible to model soft contact in MuJoCo, which can be a topic of exploration.

1.3 The Grasp Matrix

The grasp matrix $\mathbf{G} \in \mathbb{R}^{6 \times n}$, where n is the number of wrench directions, defines how forces exerted at the contact points are mapped into wrenches. You may remember from class or the homework that we define $\mathbf{G}$ to contain wrenches in the body frame, rather than the contact frame. We can convert the basis, B_{c_i} from the contact frame to the object frame applying the following transformation:

$$F_o = Ad_{g_{oc_i}}^{T-1} B_{c_i},$$

Where,

$$Ad_{g_{oc_i}}^{T-1} = \begin{bmatrix} R_{oc_i} & 0 \\ \hat{p}_{oc_i} R_{oc_i} & R_{oc_i} \end{bmatrix}$$

This would be useful to think about if you want to represent wrenches in the object frame. MuJoCo, however, automatically gives you each contact in the *global* frame. It turns out that you can create a valid grasp matrix as long as all of your contacts are in the same frame. Thus you might not have to do this conversion in practice.

Another way to think about the grasp matrix is by considering the following wrench basis for a point contact:

$$\mathbf{G}\mathbf{l} = \begin{bmatrix} \mathbf{f}_{c_1} & \mathbf{f}_{c_2} & \mathbf{f}_{c_3} \\ (\mathbf{c}_1 - \mathbf{o}) \times \mathbf{f}_{c_1} & (\mathbf{c}_2 - \mathbf{o}) \times \mathbf{f}_{c_2} & (\mathbf{c}_3 - \mathbf{o}) \times \mathbf{f}_{c_3} \end{bmatrix}$$

Given a set of n_c contact points, each defines its own wrench matrix and basis. The grasp matrix $\mathbf{G}$ is the matrix obtained by composing the various wrench bases side by side. The grasp matrix $\mathbf{G}$ can be seen as a linear map $\mathbf{G} : \mathbb{R}^n \rightarrow \mathbb{R}^6$ mapping the intensity of forces and moments into the overall wrench acting on $\mathbf{B}$. Vector $\mathbf{l} \in \mathbb{R}^n$, where n is the number of contacts, contains the magnitude of force in for each wrench in the grasp matrix.

1.3.1 Friction Cone Discretization

The friction cone described earlier is a useful tool to determine the properties of a grasp. In practice, however, we cannot represent this cone with an infinite number of vectors. Thus the circular friction cone can be approximated by a cone generated by a finite set of vectors. Figure 5 shows an illustration of this.

Using this approximation, each contact force lying within the friction cone can be written as a non-negative combination of forces along the boundary of the friction cone, i.e., $\mathbf{d}_i = \sum_{j=1}^k \alpha_i^j \mathbf{d}_i^j$ with $\alpha_i^j \geq 0$. Based on this discretization the wrench generated by the i th contact force can then be written as:

$$\begin{aligned} \mathbf{w}_i &= \begin{bmatrix} \mathbf{d}_i \\ (\mathbf{c}_i - \mathbf{o}) \times \mathbf{d}_i \end{bmatrix} = \begin{bmatrix} \sum_{j=1}^k \alpha_i^j \mathbf{d}_i^j \\ (\mathbf{c}_i - \mathbf{o}) \times \sum_{j=1}^k \alpha_i^j \mathbf{d}_i^j \end{bmatrix} = \\ &= \sum_{j=1}^k \alpha_i^j \begin{bmatrix} \mathbf{d}_i^j \\ (\mathbf{c}_i - \mathbf{o}) \times \mathbf{d}_i^j \end{bmatrix} = \sum_{j=1}^k \alpha_i^j \mathbf{w}_i^j \end{aligned}$$

Where the kn wrenches $\mathbf{w}_j^i$ can be arranged into a $6 \times kn$ dimensional matrix $\mathbf{G}$, which approximates the grasp matrix.

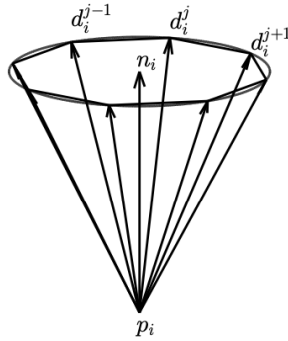


Figure 5: Friction cones are commonly approximated with regular pyramids

1.4 Force Closure

Intuitively, a grasp achieves force closure if it is possible for it to be maintained in despite any external force acting on the restrained object (assuming the grasp can exhibit the necessary counter forces). This idea can be rigorously formalized, but it should be immediately noticed that there exist slightly different definitions for this concept.

A concise definition of force closure can be given using the grasp matrix $\mathbf{G}$. A grasp achieves force closure if it can balance any external wrench $\mathbf{w}_e$ applied to the object. Algebraically, this means that for each external wrench $\mathbf{w}_e$, there exist a vector $\mathbf{l} \in FC$ such that

$$\mathbf{G}\mathbf{l} = -\mathbf{w}_e, \mathbf{w}_e = \sum_{i=1}^n \sum_{j=1}^k \alpha_i^j \mathbf{w}_i^j$$

From this algebraic definition it is possible to immediately derive algebraic conditions to determine if a given grasp achieves force closure. For example, a necessary condition is that $\text{rank}(\mathbf{G}) = 6$. However this condition is just necessary but not sufficient, because $\mathbf{l}$ is restricted to be in the friction cone FC , and this is a subset of $\mathbb{R}^n$. A necessary and sufficient condition is that $\mathbf{G}(FC) = \mathbb{R}^6$, where $\mathbf{G}(FC)$ is the set obtained by applying the grasp map to every point in the friction cone FC .

An equivalent necessary and sufficient condition for force closure that will be useful for us in the next section is:

$$\mathbf{0} \in \text{int}(\text{co}\{\mathbf{w}_1^1 \dots \mathbf{w}_n^m\})$$

Where $\text{co}(\cdot)$ and $\text{int}(\cdot)$ denote the convex hull and the interior of a set, respectively, and $\mathbf{0}$ is the origin of the wrench space.

1.5 Grasp Synthesis Algorithm

In order to automatically generate a robust grasp, we must do two things: bring the fingers to feasible points on the object and adjust them to achieve force closure. In this section we outline a method to achieve both of these within a gradient-based optimization problem heavily drawn from [3]. A nice side effect we get from this is being able to evaluate whether a grasp is in force closure, which we'll also learn how to do.

The novel force closure test proposed by the authors in [3] is based on the Q norm $\|\cdot\|_Q$, a special norm different from the commonly used $L2$ norm. Starting from this special norm (see [3] for details), two distances between a point p and a convex polyhedron A are defined:

$$\begin{aligned} d_Q^+(\mathbf{p}, A) &= \min_{\mathbf{a} \in A} \|\mathbf{a} - \mathbf{p}\|_Q \\ d_Q^-(\mathbf{p}, A) &= -\min_{\mathbf{a} \in \partial A} \|\mathbf{a} - \mathbf{p}\|_Q \end{aligned}$$

Remember from the last section that our aim is to ensure that $\mathbf{0} \in \text{int}(\text{co}\{\mathbf{w}_1^1 \dots \mathbf{w}_n^m\})$. In [3], the authors prove that $d_Q^+(0, \text{co}\{\mathbf{w}_1^1 \dots \mathbf{w}_n^m\}) = 0$ if and only if $\mathbf{0} \in \text{co}\{\mathbf{w}_1^1 \dots \mathbf{w}_n^m\}$. This condition is necessary but not sufficient to ensure force closure, meaning it must be true to guarantee force closure, but it doesn't guarantee force closure on its own. The reason for this is that the zero wrench could still be on the boundary of $\text{co}\{\mathbf{w}_1^1 \dots \mathbf{w}_n^m\}$ and not the interior. However, a necessary and sufficient condition can be determined using d_Q^- . This test is in essence a test on the sign of d_Q^- , i.e., the grasp achieves force closure if and only if $d_Q^- < 0$. The grasping planning algorithm is then implemented by an iterative procedure where $dQ(q)$ is a case-by-case function defined as dQ^+ or dQ^- .

1.5.1 The Necessary Condition

For this project we can use a similar idea, with some simplifications. First, since our geometry is simple enough, we can use a standard $L2$ norm instead of the Q norm. Therefore, our objective is the following:

$$\begin{aligned} d_{L2}^+(0, W) &= \min_{\alpha \geq 0} \|\mathbf{G}\alpha\|_2 + \beta D \\ \text{s.t. } &\sum_{i=1}^N \alpha_i = 1 \end{aligned}$$

Where $D = d_1^2 + d_2^2 + d_3^2 + d_4^2 \in \mathbb{R}$ is the sum of squared distances, d_i , from finger i to the surface of the object, and β is a distance weight hyperparameter. Conceptually, this optimization is minimizing the distance between the origin in wrench space and the set $W(\mathbf{q}) = \{\mathbf{G}\alpha | \alpha \geq 0\}$ where $\mathbf{q}$ is some vector in $\mathbb{R}^n$. This is roughly equivalent to the $d_Q^+(0, \text{co}\{\mathbf{w}_1^1 \dots \mathbf{w}_n^m\}) = 0$ condition we saw before. For the sake of terminology, we'll call $d_2^+(0, W)$ the Q+ distance. Now we still have to ensure that the wrench origin is not on the boundary of this set. We'll take a look at that next.

1.5.2 The Sufficient Condition

Remember that according to [3], $d_Q^- < 0$ is sufficient for force closure. This definition is nebulous so far, but how we solve that has an intuitive explanation. The problem of calculating d_Q^- is equivalent to finding the largest ball around our wrench space origin that is also in the wrench space. If the radius, r of this ball is > 0 , then the origin in wrench space is in the interior of our set, and not the boundary. In this setup, $r > 0$ implies $d_Q^- < 0$. In order to calculate the sufficient condition, we solve the following optimization problem:

$$\begin{aligned} d_Q^-(k) &= \min -r, \\ \text{s.t. } r\mathbf{q}_k &= \sum_{i=1}^N \alpha_i \mathbf{g}_i, \\ \sum_{i=1}^N \alpha_i &= 1, \\ \alpha_i &\geq 0, r \geq 0. \end{aligned}$$

Where what we'll call the Q^- distance is defined as:

$$d_Q^-(\mathbf{G}) = \max_{k=1, \dots, K} d_Q^-(k).$$

In the above equations, $\mathbf{q}_k \in \mathbb{R}^6$ is a unit vector in some random direction, and $\mathbf{g}_i$ is the i 'th column of $\mathbf{G}$. The intuition for $d_Q^-(\mathbf{G})$ is that we find the solution to $d_Q^-(k)$ that is the closest to zero, where the solution to $d_Q^-(k)$ represents the largest length along direction $\mathbf{q}_k$ that is still in W , which satisfies our constraints. Since it's intractable to do this for every direction in the unit ball, we sample K directions as an approximation.

1.5.3 The Algorithm

Now we're ready to put this theory together in an algorithm. Algorithm 1 shows the grasp synthesis algorithm, and algorithm 2 shows the method that optimizes the objective function. The update rule is the following:

$$\mathbf{q}_h^{new} = \mathbf{q}_h - \nabla_{\mathbf{q}_h} f,$$

Where,

$$f = \begin{cases} d_{L2}^+(0, W), & \mathbf{0} \notin \text{co}\{W\} \\ d_Q^-(\mathbf{G}), & \mathbf{0} \in \text{co}\{W\} \end{cases}$$

1.5.4 Grasp Synthesis Results

After correctly implementing grasp synthesis, you should see that the resulting grasp achieves force closure mathematically and that the calculated Q^+ value is essentially zero while the Q^- value is negative. In your report, you should show the Q^+ and Q^- values for grasps from different heights of the palm. What you should see is that the grasp synthesis algorithm is unable to find a force closure grasp when the palm is too high, since there are no forces pushing up into the ball. **Note**, however, that having a grasp that is mathematically in force closure doesn't guarantee that the hand will pick it up in the control loop. This is because force closure only works when there are sufficient forces being applied in that exact grasp. When using the control loop, other factors, such as hand PD values, friction, mass, physics solver quirks, etc. all affect whether your grasp works. Note that it's also very much possible to pick up the ball successfully without achieving force closure at all. For the purpose of this project, we want you to show that your math works using Q^+ and Q^- values and then figure out the simulator tricks to get the hand to actually pick up the ball. Try to see how high you can push the mass of the ball and how low you can bring down the friction of the ball while still being able to pick it up.

Algorithm 1 Grasp Synthesis Algorithm

Require: $\mathbf{q}_h$, max_iter, λ
iter $\leftarrow$ 0
 $\mathbf{q}_h \leftarrow$ init_configuration
in_contact = False
while iter < max_iter **do**
 if all fingers touching **then**
 made_contact $\leftarrow$ True
 end if
 $f \leftarrow$ joint_space_objective
 $\mathbf{q}_h^{new} = \mathbf{q}_h - \lambda \nabla_{\mathbf{q}_h} f(\mathbf{q}_h)$

 improvement $\leftarrow f(\mathbf{q}_h) - f(\mathbf{q}_h^{new})$
 if improvement > 0 **then**
 $\mathbf{q}_h \leftarrow \mathbf{q}_h^{new}$
 if improvement < $1e^{-6}$ **then**
 break
 end if
 end if
end while
return $\mathbf{q}_h$

Algorithm 2 Joint Space Objective

Require: $\mathbf{q}_h$, β , fingertip_positions, in_contact, Q_plus_thresh
 $D =$ get_total_dist_from_sphere(positions)
if not in_contact **then return** βD
else
 normals $\leftarrow$ get_contact_normals()
 $FC \leftarrow$ build_discrete_friction_cone(normals)
 $G \leftarrow$ build_grasp_map(FC)
 fc_loss $\leftarrow$ optimize_necessary_condition(G)
 if fc_loss < Q_plus_thresh **then**
 fc_loss $\leftarrow$ optimize_sufficient_condition(G)
 end if
 return fc_loss + βD
end if

1.6 Levenberg-Marquardt Inverse Kinematics

Inverse kinematics is the process of mapping the end-effector’s pose (position and orientation) to the joint coordinates that will achieve that pose.

You may remember from EECS 106A that we solved inverse kinematics by breaking down rotations of revolute joints into Paden–Kahan subproblems. While this may work well for more simple robots, things get more complicated when we work with multifingered hands. Often, in practice, IK solvers instead use numerical methods to reach solutions. One of your tasks in this project will be to implement multifingered IK using a well-known numerical method called Levenberg-Marquardt.

The Levenberg-Marquardt (LM) method, also known as the damped least squares method, is primarily used to solve non-linear least squares problems. The equation for LM is the following:

$$q_{t+1} = q_t + ((J^T J + \lambda I)^{-1} J^T (x_d - x_t))$$

Where q_t is the joint state at timestep t , J is the manipulator Jacobian, λ is the damping coefficient, x_d is the desired pose, and x_t is the target pose at timestep t . In your implementation loop you will iterate on this equation until your current pose is within some threshold of the desired pose.

This [article](#) does a great job explaining the basic theory and how to implement it in MuJoCo. Note, however, that this article explains how to solving IK for one end effector. In this project, you will have to implement IK that solves this equation for four fingers at the same time. Luckily, the main algorithm is mostly the same, and you can concatenate the manipulator Jacobians for each finger.

1.7 Other Grasping Methods

1.7.1 Model-less Algorithms

In this project we primarily dealt with model-based algorithms, in which the object geometry is known a-priori. A significant body of recent research on grasping considers the situation where a geometric model of the object to be grasped is not available upfront. This problem domain is considered to be much more challenging, but it also has a much broader applicability than model-based algorithms and presents the opportunity for broader application sets. To remedy the lack of an a priori model, these grasping algorithms rely on sensor data acquired at run time. Typically, these include vision and tactile sensing. Information acquired at run time could then be used to build an approximate or incomplete object model on the fly, or to compute a grasping strategy directly from raw data without attempting to produce a model. One could retort that an algorithm that builds an approximate model on-the-fly is not model-less anymore. Here we consider an algorithm model-less if it does not assume the availability of a model a priori. Then, according to this definition, an algorithm building a model on-the-fly is still model-less. It goes without saying that the use of machine learning algorithms is prevalent in this area.

1.7.2 Dual Arm Grasping

Various technological developments and practical reasons continue to drive research in dual arm grasping and manipulation. From a practical point of view, certain grasping tasks require interaction with larger sized parts that may have physical properties (weights, inertias..) and manipulation requirements that would require two or more robotic arms utilizing simple grippers or even possibly robotic hands. In these scenarios, the use of two arms is then preferred. Moreover, there is an increasing expectation that robots will work side-by-side with humans in a variety of everyday tasks (corobots). In this case robots have to be able to use the same tools and operate in the same environments designed for humans. As these designs often assume end-users are equipped with two hands, a dual arm design will then ease the integration of robots into these environments. Finally, continued research in humanoid robotics is naturally linked to dual arm grasping and manipulation.

1.7.3 Machine Learning Approaches

There are two main ways in which machine learning has been applied to grasping. The first relies primarily on vision using RGB and depth information from cameras to generate a full or partial model of the object at run-time as input to the model which then generates feasible grasps on this object. Some notable examples of this using parallel-jaw grippers (the grippers currently equipped on the Sawyers) are [4], [5] with a recent approach that includes

multi-fingered hands being [6].

The second approach focuses more on in-hand manipulation of the object which can be done using solely tactile feedback and proprioception of the hand with the object [7], [8] these agents are usually trained in simulation first with reinforcement learning and then applied to real. If you're curious about more work like this we highly recommend checking out the fantastic work of [Haozhi Qi](#) a PhD here at Berkeley!

2 Project Tasks

In this project, you will be implementing an algorithm to get an Allegro hand attached to a Sawyer to grasp and pick up a ball. You will also have an opportunity to experiment with other aspects of MuJoCo and grasping.

1. **Multifingered Inverse Kinematics:** Using the Levenberg-Marquardt method, implement the inverse kinematics solver in `multifingered.ik.py` then demonstrate the following tasks:
 - Demonstrate your implementation by running IK on the following values:
 - `body_names` = ['sawyer/allegro_right/palm', 'sawyer/allegro_right/ff_tip_rubber', 'sawyer/allegro_right/mf_tip_rubber', 'sawyer/allegro_right/rf_tip_rubber', 'sawyer/allegro_right/th_tip_rubber']
 - `body_positions` = [[1.0139, -0.4455, 1.4342], [1.0566, -0.3681, 1.4689], [1.0564, -0.3731, 1.4255], [1.0765, -0.3651, 1.3784], [0.9403, -0.4003, 1.6041]]
 - `body_orientations` = [[-0.62, -0.588, -0.3918, -0.3287], [0.3301, 0.305, -0.6762, -0.5838], [0.3465, 0.2863, -0.6522, -0.6104], [0.0174, -0.0705, -0.7239, -0.686], [-0.9672, 0.061, 0.0294, 0.2446]]
 - In your report answer the following question:
 - One naive way to synthesis a grasp on a ball is to choose four random points on the ball and use IK to bring the fingers to these points. Why is this not guaranteed to work? What can you do instead?
2. **Grasp Synthesis:** Implement the algorithm in section 1.5.
 - Demonstrate the following procedure in a single video:
 - Initialize your grasp to start in your calculated force closure grasp (or optionally, have the hand start from above and come into the force closure grasp)
 - Lift the ball vertically to an ending position.
 - Note that you will have to control the robot. Consider interpolating between your current desired joint angles based on a timestep and setting `physics.data.ctrl` to those joint positions at every timestep.
 - Experiment with varying the grasp height (z position of the palm) and determine how your calculated $Q+$ and $Q-$ distances relate to grasp success.
3. **Extensions [EXTRA CREDIT]:** You may implement at least **one** of the following extensions of your choice list below. Each extension can get you at most **3** points. You may implement as many extensions as you like, but at most **three** extensions will be counted for extra credit for a maximum potential of **9** points.
 - Demonstrate task 2 on another shape (cube, pyramid, mesh of a custom object, etc.).
 - Perform grasp quality evaluation analysis using a method such as [Ferrari-Canny](#) or others (robust force closure, gravity resistance, etc.).
 - Implement palm pose optimization outlined in [9] to incorporate palm position into the grasp planning.
 - Experiment with alternative types of control, such as force control
 - Experiment with different attributes for the ball (size, mass, friction, stiffness, damping, etc.).
 - Make the robot pick up an object and place it into a container
 - Experiment with implementing a soft contact model
 - Create a trajectory for your hand motion
 - An idea of your choice. However, you *must* consult a staff member for your idea to be approved.

Note about extensions: We are deliberately making this section very open ended. We are expecting that you formulate each extension as an experiment and provide quantitative metrics that demonstrate how your method is working.

3 Deliverables

To demonstrate that your implementation works, deliver the following in a report. The purpose of these project reports are to gradually scale up to a full conference-style paper, which you'll be writing for your final project. Please format your report using the IEEE two-column conference template.

1. Abstract: An abstract, of at most 150 words, describing what you did and what results you achieved. Conveying information concisely is a difficult skill, and we want you to practice it here.
2. Methods: Explain in detail your approach towards each task in the previous section, making sure to address the following questions. You can expect the readers of your report to know everything in this manual, so you don't need to repeat any of it (though you should reproduce equations and the like as needed for readability).
 - (a) For the IK solver, how did you adapt the algorithm to work for multiple fingers?
 - (b) What's your answer to this question: One naive way to synthesis a grasp on a ball is to choose four random points on the ball and use IK to bring the fingers to these points. Why is this not guaranteed to work? What can you do instead?
 - (c) Describe how the grasp synthesis algorithm works including the objective function and how it optimizes for force closure.
 - (d) Explain at least one of your design decisions in this project. Why did you choose it? What are possible alternatives?
 - (e) **Extra Credit:** Which extensions did you choose and how did you approach them?
3. Experimental results: Summarize the performance of each project task. For each point provide a very brief comment or analysis of the result.
 - (a) For IK show an image of the result of solving IK on the values provided in task 1 (**Hint:** it should look like a thumbs up).
 - (b) For the grasp planner plot the loss curve for the objective over time
 - (c) Provide images that show the hand in the following stages: before the grasp, after fingers made contact (but before the adjustment), after adjusting for force closure
 - (d) For your grasping algorithm provide a table showing your Q+ and Q- distances for at least five starting heights of the palm, and the success of the grasp (fail, partial success, success)
 - (e) For your grasp provide the following plots for both a grasp that worked and one that didn't:
 - Contact force vs. Time
 - Number of Contacts vs. Time
 - (f) Provide the values for the lowest friction and highest mass of the ball that you were able to successfully pick up
 - (g) **Extra Credit:** For your extensions provide plots or tables of your choosing. **Note:** we're leaving this part very open ended. This means we're not expecting anything specific, but we do expect you to provide quantitative figures for the experiments you chose.
4. Discussion:
 - Discuss the performance of each project task. What worked, what didn't, and why?
 - What worked well with the grasp synthesis algorithm and what didn't. How might you improve this algorithm?
 - Describe any challenges you had in this project and how you solved them. If you couldn't solve them, why?
 - What future work could be possible as an extension to this project? You can reference section 1.8.
5. Bibliography: A bibliography section citing any resources you used. This should include any resources you used from outside this class. Please use the [IEEE citation format](#).
6. Appendix:

- (a) GitHub Link: Provide the link to your GitHub classroom repo. Simply push your code to the private repository that was created for your team, and we will be able to see any changes you push to your assignment repository.
 - (b) Provide videos of the IK task and the full grasping task. Also include addition videos as necessary for your extensions.
7. Page Limit: To prepare you for conference-style papers (and to protect the graders' time), reports are **limited to 6 pages**, not including the bibliography or appendix.
- (a) While we are not enforcing a figure limit, we expect your figures to be concise, informative, and readable, with detailed captions for each. We expect you to need about 10-13 figures. It may be helpful to combine some of the pictures in the report into one for easy comparison.

4 Getting Started

Please pull from the same repo as the rest of the projects. Since this project is mainly in simulation you may do it on your personal computer or any of the lab workstations.

We suggest that you start by following this official [tutorial](#) on MuJoCo. This can be a bit of a learning curve, but once you get the hang of it, you can do some pretty powerful stuff! You can access the full documentation [here](#).

We will also be using a wrapper on MuJoCo called [dm_control](#), by Google Deepmind. We suggest also following their tutorial [here](#).

4.1 GitHub

We will use Github classroom. Join link <https://classroom.github.com/a/dw3vJxGr>

4.2 Starter Code

Pull the starter code!

Then run the following commands:

```
cd project4a
git submodule init && git submodule update
```

We will use uv for package management this project. Run the following:

```
distrobox enter ros2
curl -LsSf https://astral.sh/uv/install.sh | sh
```

In a NEW terminal run the following:

```
distrobox enter ros2
cd project4a/
uv venv -p 3.12
uv sync
```

This should complete setup! Now in main.ipynb you just need to select .venv as your kernel.

The starter code is meant to serve as a starting point for your implementation, but it is just a suggestion. You may change any part of the code that you'd like as long as you complete the deliverables.

1. `main.ipynb`: This is the main notebook in which you can test your implementation, print variables, and visualize scenes
2. `multifingered_ik.py`: This is where you can implement your IK solver. We provide some skeleton code here.
3. `AllegroHandEnv.py`: Use can use this environment to configure the Allegro hand grasping environment.

4. `grasp_synthesis.py`: This is where you can implement your grasp synthesis algorithm. We provide some skeleton code here.
5. `mujoco_menagerie`: This folder contains MJCF (MuJoCo's version of the URDF format) of many commonly used robot models. We will mainly be using the Sawyer and Allegro hand models, but feel free to look around if you're interested. You are free and encouraged to *modify* the MJCF files for the Sawyer or Allegro models.

Modifying Code: The starter code in this project is a suggestion and you may modify anything you need to. Even though you'll see `#YOUR CODE HERE`, you may or may not choose to write code there as long as you still complete the deliverables.

4.3 Implementation Tips

- Defining helper functions is a good software engineering skill that can help you minimize redundant code.
- If you want to access the MuJoCo model format for your `dm_control` model, use `physics.model.ptr`, and for data use `physics.data.ptr`.

START EARLY! We expect this project to be the most intensive out of all projects so far.

4.4 Common Issues/Concerns

- Check out the [Mujoco tips document](#) for updated tips:
- Contacts and slip can be tricky to deal with in MuJoCo. Consider playing around with parameters such as object mass, friction, size, etc.
- When determining contact positions and normals, consider any contact between the ball and the hand (don't include the contact between the ball and the floor)
- We recommend pre-computing all of the grasp positions that you'll need in the control loop. For instance, compute the grasp qpos and any intermediate positions before the control loop, then interpolate between positions based on the step, and set `physics.data.ctrl` to the desired position at that step.
- For project task 2, grasp synthesis, we will accept pre-setting the robot configuration to the grasp position outputted by `synthesize_grasp()` and having the control loop pick the object up from there.
- For project task 1, select the grasp points on the ball however you want. This part is not supposed to work. You should find that it's difficult to set four grasp points that are feasible for the hand as long as it obeys the joint limits. That's all we want to see, and task 2 is the solution to this problem.
- There's a difference between setting qpos and actually simulating physics. Simulation only happens in the control loop when you call `physics.step()`. Calculate your desired qpos's from IK or grasp synthesis before your control loop, then set `physics.data.ctrl` to interpolated positions.

5 Scoring

Summing all this up, this project will be out of 60 points, with an additional 9 points possible.

6 Submission

You'll submit your writeup on Gradescope and push your code to GitHub. Your code will be checked for plagiarism, so please submit your own work. Only one submission is needed per group. Please add your groupmates as collaborators on both GitHub and Gradescope.

7 Improvements

If you notice any typos or things in this document or the starter code which you think we should change, please let us know. Next year's students will thank you.

Table 1: Point Allocation for Project 4

Section	Points
Figures: Quality and Readability	5
Abstract	5
Methods: IK solver	10
Methods: Grasping algorithm	10
Methods: Extensions Extra Credit	9
Results	10
Discussion	10
Bibliography	4
Code	3
Video	3

8 References

- [1] R. M. Murray, S. S. Sastry, and L. Zexiang. *A Mathematical Introduction to Robotic Manipulation*. 1st. USA: CRC Press, Inc., 1994.
- [2] S. C. Carpin et al. “Multi-Fingered Robotic Grasping: A Primer”. *Cornell University Library* (2016).
- [3] X. Zhu and J. Wang. “Synthesis of Force-Closure Grasps on 3-D Objects Based on the Q-Distance”. *IEEE Transactions on Robotics and Automation* (2003).
- [4] J. Mahler et al. “Dex-Net 1.0: A cloud-based network of 3D objects for robust grasp planning using a Multi-Armed Bandit model with correlated rewards”. *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016.
- [5] M. Sundermeyer et al. “Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes” (2021).
- [6] J. Zhang et al. “DexGraspNet 2.0: Learning Generative Dexterous Grasping in Large-scale Synthetic Cluttered Scenes”. *8th Annual Conference on Robot Learning*. 2024.
- [7] H. Qi et al. “In-Hand Object Rotation via Rapid Motor Adaptation”. *Conference on Robot Learning (CoRL)*. 2022.
- [8] H. Qi et al. “General In-Hand Object Rotation with Vision and Touch”. *Conference on Robot Learning (CoRL)*. 2023.
- [9] Y. Fan et al. “Real-Time Grasp Planning for Multi-Fingered Hands by Finger Splitting”. *IROS* (2018).